Full Stack Development with MERN: Project Documentation

1. Introduction

- **Project Title:** HouseRent [StayHub - House & Commercial Rental Platform]

2. Project Overview

- **Purpose:** StayHub simplifies property renting and selling by connecting property owners directly with prospective tenants through a streamlined, secure web application.
- **Features:**
    - Role-Based Access Control (Owner & Renter dashboards).
    - Direct property listing management with dynamic image previews and Base64 persistent storage.
    - Real-time property searching and filtering by address, listing type (Rent/Sale), and category (Residential/Commercial).
    - Booking request workflows with custom tenant contact details and move-in date selection.
    - Owner booking management (Accept/Toggle booking status).

3. Architecture

- **Frontend:** Built with React (Vite), React Router DOM for client-side routing, and Axios for RESTful API communication with customized auth headers.
- **Backend:** REST API built using Node.js and Express.js, featuring modular MVC architecture (Controllers, Routes, Models, and Middleware).
- **Database:** MongoDB Atlas utilizing Mongoose schemas with data relationships (ObjectId referencing between Users, Properties, and Bookings).

4. Setup Instructions

- **Prerequisites:** Node.js (v18+), npm, MongoDB Atlas Account or Local MongoDB instance.
- **Installation:**

❖ Setup Backend:

cd backend

npm install

❖ Setup Frontend:

cd ../frontend

npm install

5. Folder Structure

- **Client (frontend/):**

o frontend/
o public/
o src/
➢ components/ # Navbar, ProtectedRoute, etc.
➢ pages/ # Home, Login, Register, OwnerDashboard, RenterDashboard, PropertyDetails.
➢ services/ # api.js (Axios instance)
➢ App.jsx
➢ main.jsx
o index.html
o package.json

- **Server (backend/):**

o backend/
o config/ # Database connection (db.js)
o controllers/ # authController, propertyController, bookingController
o middleware/ # authMiddleware.js
o models/ # User.js, Property.js, Booking.js

- routes/ # authRoutes, propertyRoutes, bookingRoutes, adminRoutes
- server.js # Express server entry point
- package.json

6. Running the Application

- **Backend Server:**

  cd backend

  npm.cmd run dev

- **Frontend Client:**

  cd frontend

  npm.cmd run dev

7. API Documentation

Method	Endpoint	Description	Protected
POST	/api/auth/register	Register new user (Owner/Renter)	No
POST	/api/auth/login	Authenticate user & get token	No
GET	/api/properties	Fetch all approved properties with filter query	No
GET	/api/properties/:id	Get detailed information for a single property	No
POST	/api/properties	Create a new property listing	Yes
GET	/api/properties/my-properties	Fetch listings created by the logged-in owner	Yes

Method	Endpoint	Description	Protected
POST	/api/bookings	Submit a rental booking request	Yes
GET	/api/bookings/my-bookings	Fetch booking requests sent by the logged-in renter	Yes
GET	/api/bookings/owner-bookings	Fetch booking requests received by the owner	Yes
PUT	/api/admin/bookings/:id/status	Update booking status (approved / pending)	Yes

8. Authentication

- Handled via **JSON Web Tokens (JWT)** and **bcryptjs** password hashing.
- Tokens are stored in browser localStorage upon login and transmitted via the Authorization: Bearer <token> header in HTTP requests.
- Backend middleware verifies token validity and attaches the authenticated user instance to req.user for secured endpoints.

9. User Interface

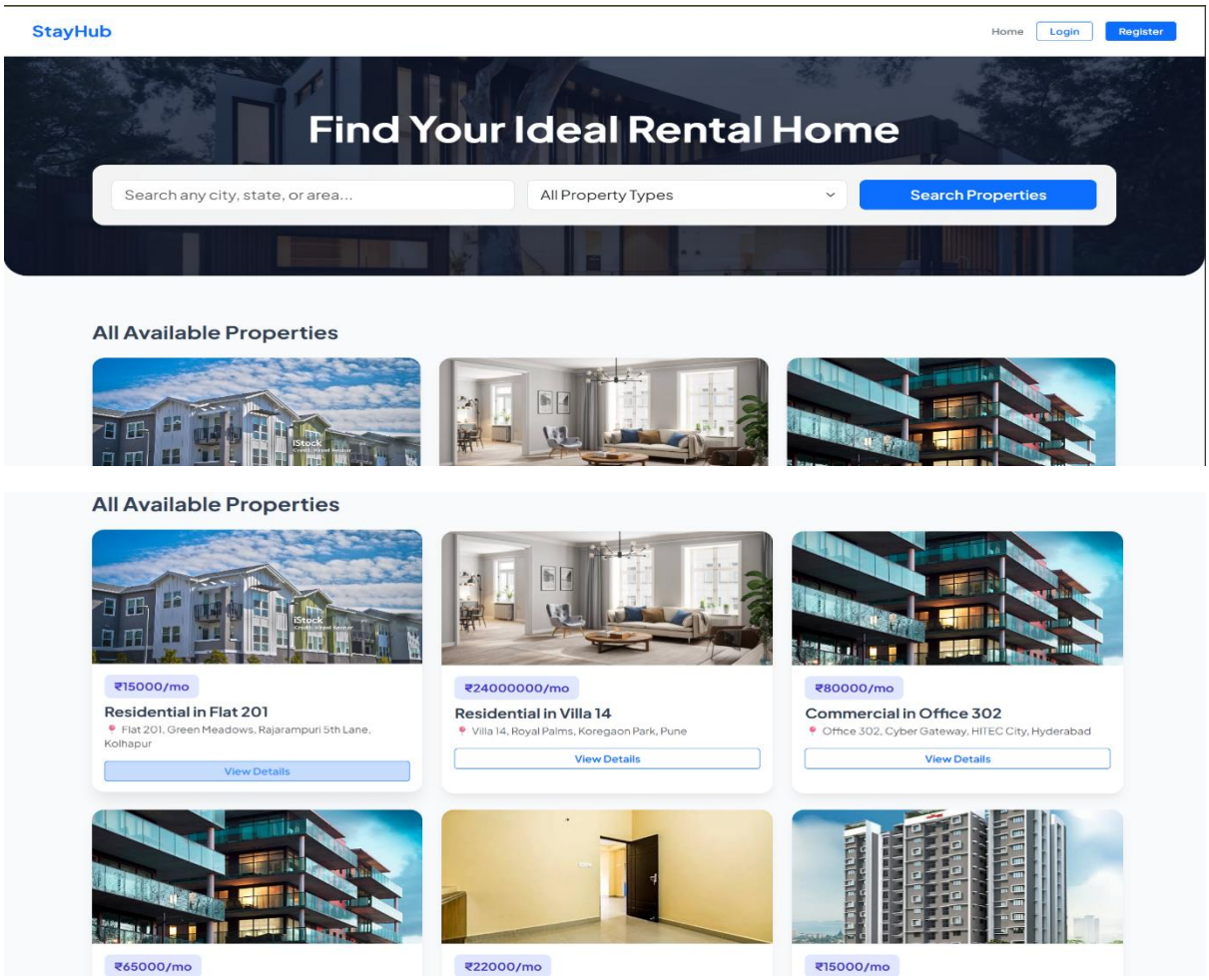
- **Home Page:** Hero search banner with dynamic filters (City, Ad Type, Property Type) and responsive listing cards.
- **Owner Dashboard:** Tabbed interface to create new listings with file uploads and track live bookings.
- **Renter Dashboard:** Browse available houses, view contact details, and inspect booking statuses.
- **Property Details View:** High-resolution preview, pricing, detailed specifications, and direct booking request form.

10. Testing

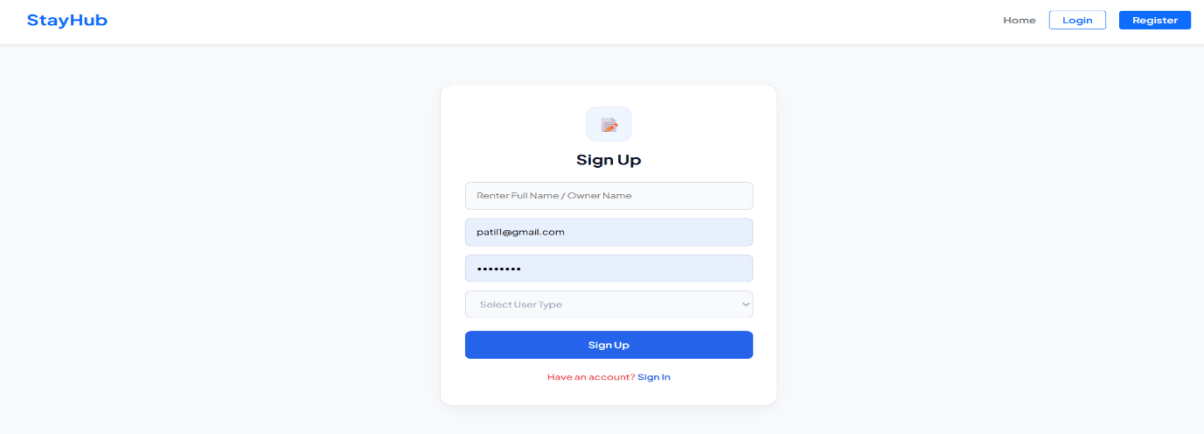
- **Manual API Testing:** Verified all CRUD routes and edge cases using Postman and Thunder Client.
- **Frontend & State Testing:** Tested authentication persistence, form validations, dynamic routing, and Base64 payload limits across Google Chrome DevTools.

11. Screenshots

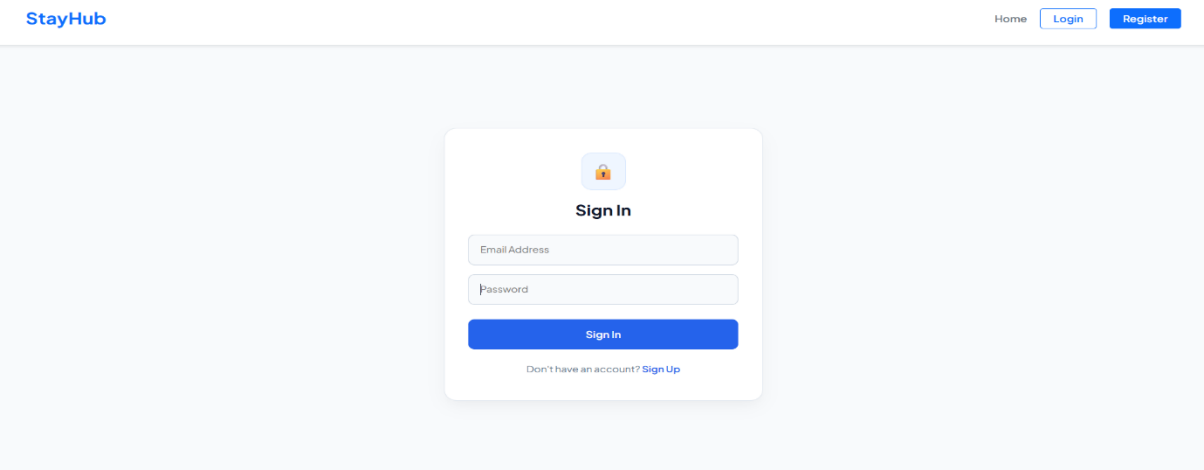
Home page :



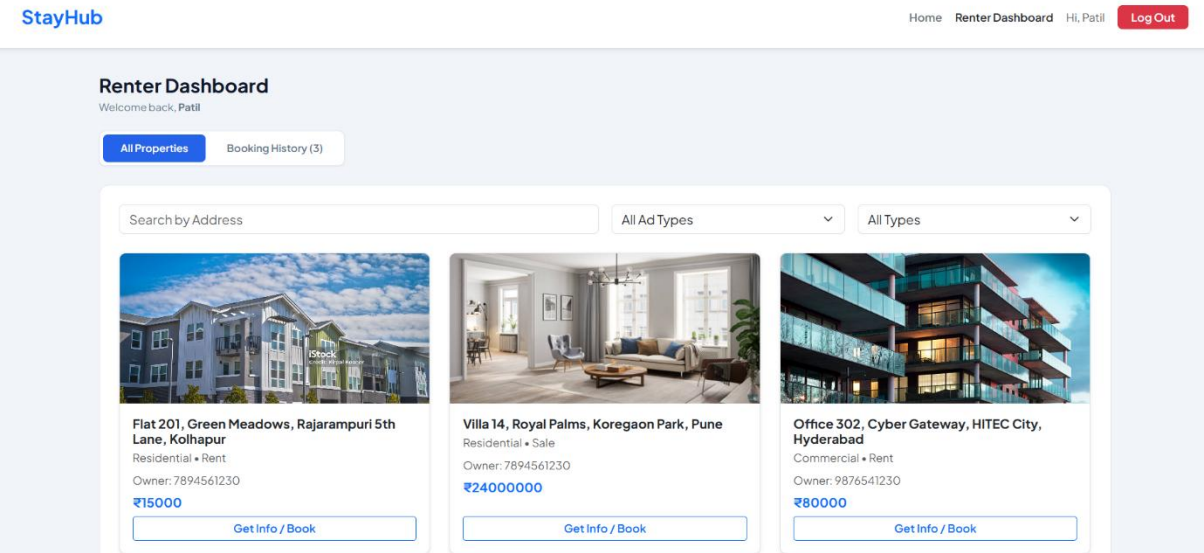
Register Page :

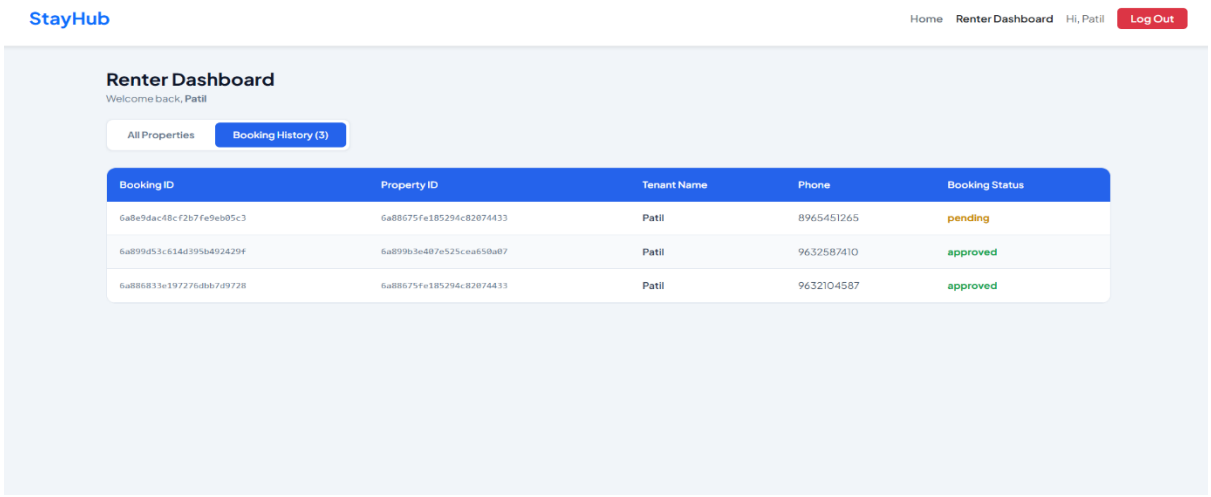


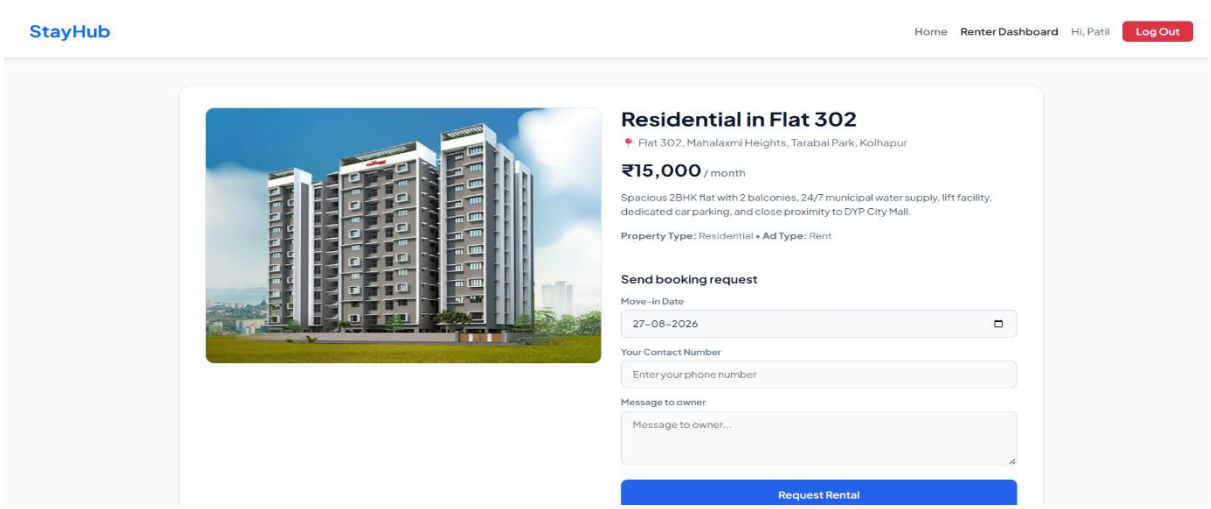
Login Page :



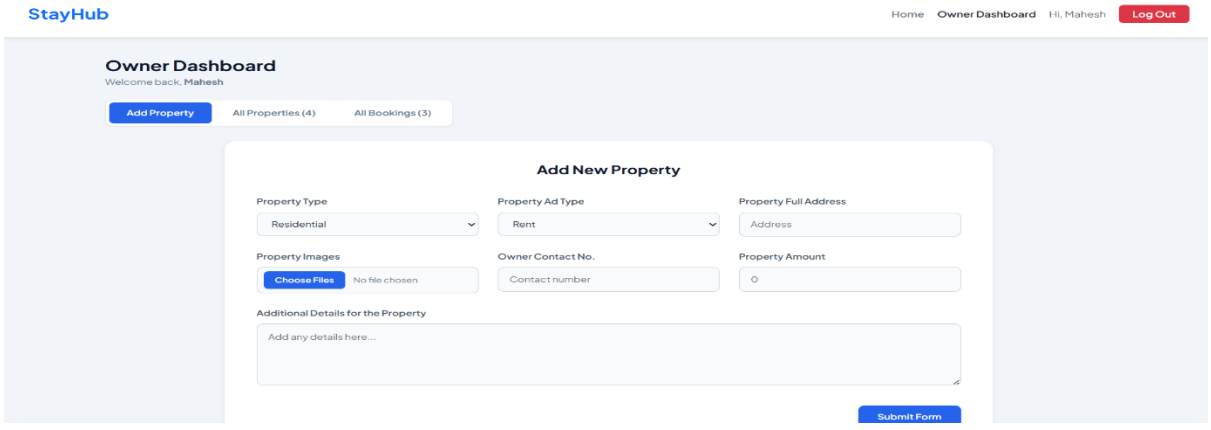
Renter Dashboard:

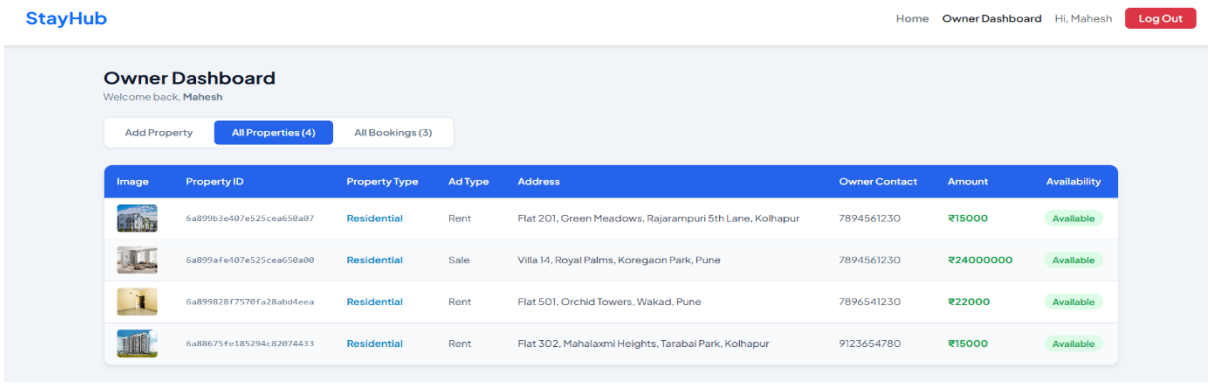


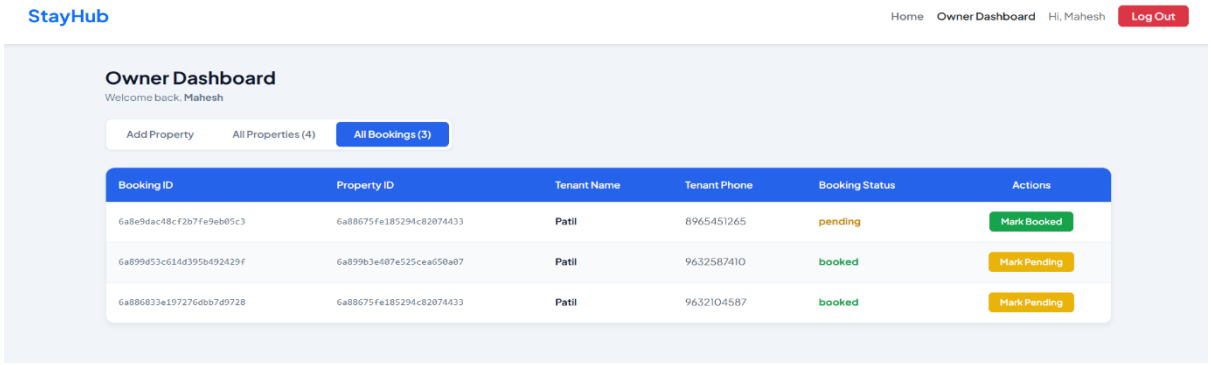




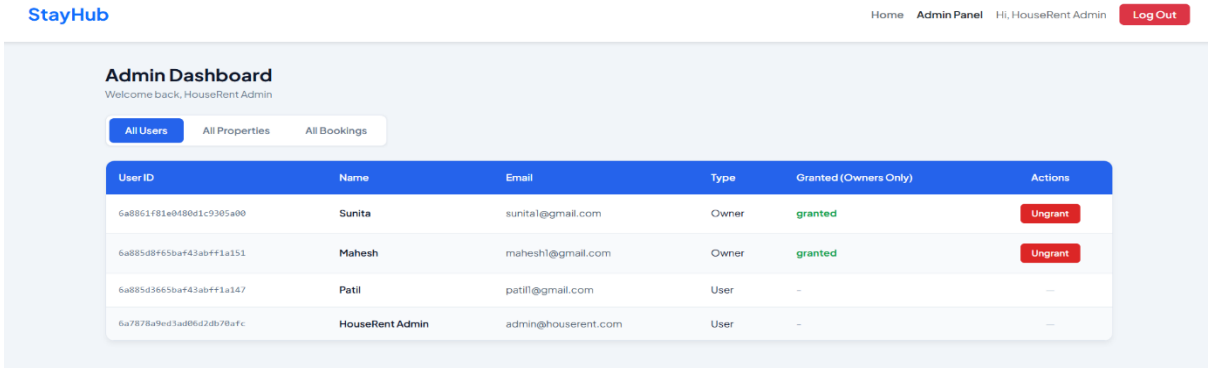
Owner Dashboard:

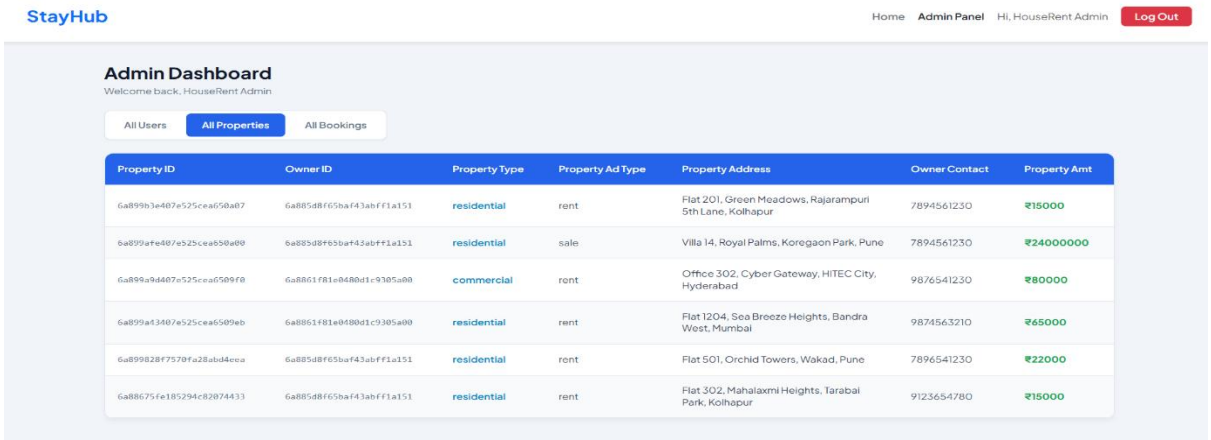


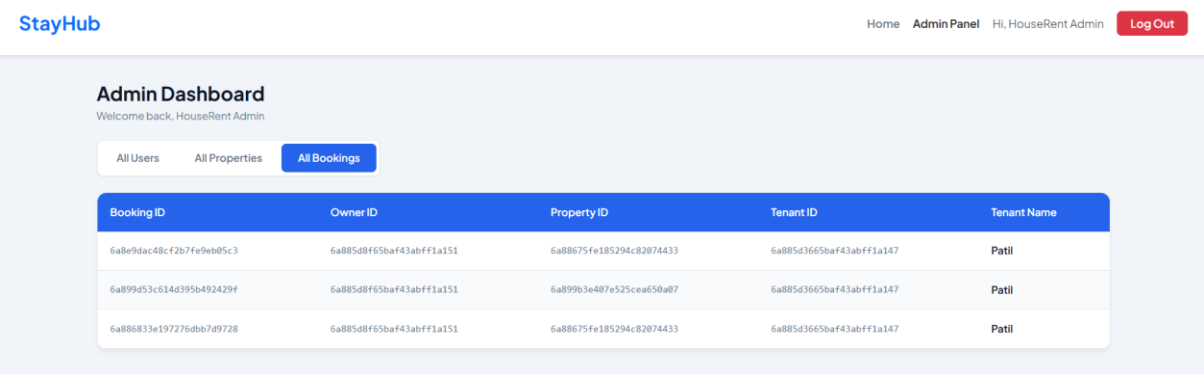




Admin Dashboard:







12. Known Issues

- Very high-resolution images stored directly as Base64 strings can increase MongoDB document size. (Resolved payload parsing limits via 50mb Express middleware).

13. Future Enhancements

- Integration with Cloudinary / AWS S3 for cloud image hosting and compression.
- Real-time in-app chat between owners and tenants using Socket.io.
- Integrated payment gateway (Razorpay / Stripe) for online token amount booking.